ROCsim - Team 1

Jiwon Lee | 251342617 | jlee3859@uwo.ca

Jacob Lee | 251362845 | jlee3887@uwo.ca

CS3307 - Deliverable 1 - Project Proposal

October 3st, 2025

1. Problem Statement*

Currently, many physics simulators are mainly optimized for NVIDIA GPUs using CUDA, but not for AMD GPUs. Having an open sourced physics simulator, optimized specifically for AMD's ROCm package, would create our application more accessible.

Our project focuses on creating a simplified version (prototype) of a real-time physics simulation, which uses ROCm (CUDA equivalent to AMD GPUs) which would run optimally on AMD's Radeon GPUs, but can still run on NVIDIA GPUs. We would reverse engineer key features from existing popular applications that use CUDA (PhysX) and implement it into our software (ROCsim). This would benefit those who want to run physics simulations locally on their AMD or NVIDIA hardware. This application is primarily based for students with a consumer based GPU who want to visualize physics concepts in real-time such as collisions, gravity, friction, particle interactions, and Hooke's Law.

Since the following project proposal is a massive undertaking, with the approval of the professor, we shall split the responsibilities into two teams. One focused on the low level and another focused on the graphical aspect. Header sections will be marked with Header* to denote that the group specific information will be highlighted in **BLUE**, i.e., that information will only be seen on the relevant team's document. Should a piece of information not be highlighted (while the header has a *), then that information is the **SAME** for both teams, i.e., the information is intercorrelated so it is repeated in both documents.

We (Team 1) would address the low level side, where we would implement a physics simulation using HIP SDK and ROCm. This would then be integrated into the OpenGL physics simulations.

2. Requirements*

Due to the significant amount of overlap for non-functional requirements, there isn't a clear way to split them between the two teams. We'll try our best to delegate responsibilities to the best of our ability.

2.1 Functional Requirements

1. **Multiple Physics Support**
    a. Support multiple physics simulation models (N-Body, collision systems, spring networks)
    b. Allow runtime switching between simulation types
    c. Support ~100-5000 particle systems with configurable parameters

2. **HIP Compute Integration**
    a. Execute physics calculations on GPU using HIP kernels
    b. Support AMD and NVIDIA GPU architectures
    c. Efficient GPU memory management and kernel execution

3. **Graphics-Compute Interoperability**
    a. Direct GPU-to-GPU data sharing between compute and rendering
    b. Minimize CPU-GPU memory transfers
    c. Synchronized compute and graphics operations

2.2 Non-functional Requirements

1. **Real-Time Performance**
    a. Maintain 30+ FPS for typical workloads
    b. Target 60 FPS for moderate particle counts
    c. Achieve significant GPU acceleration over CPU-only implementations

2. **Resource Efficiency**
    a. Optimize GPU memory usage and bandwidth
    b. Limit total system memory consumption
    c. Provide graceful degradation under resource constraints

3. System Stability
    a. Run continuously without crashes or memory leaks
    b. Handle GPU errors and edge cases gracefully
    c. Provide meaningful error messages and logging

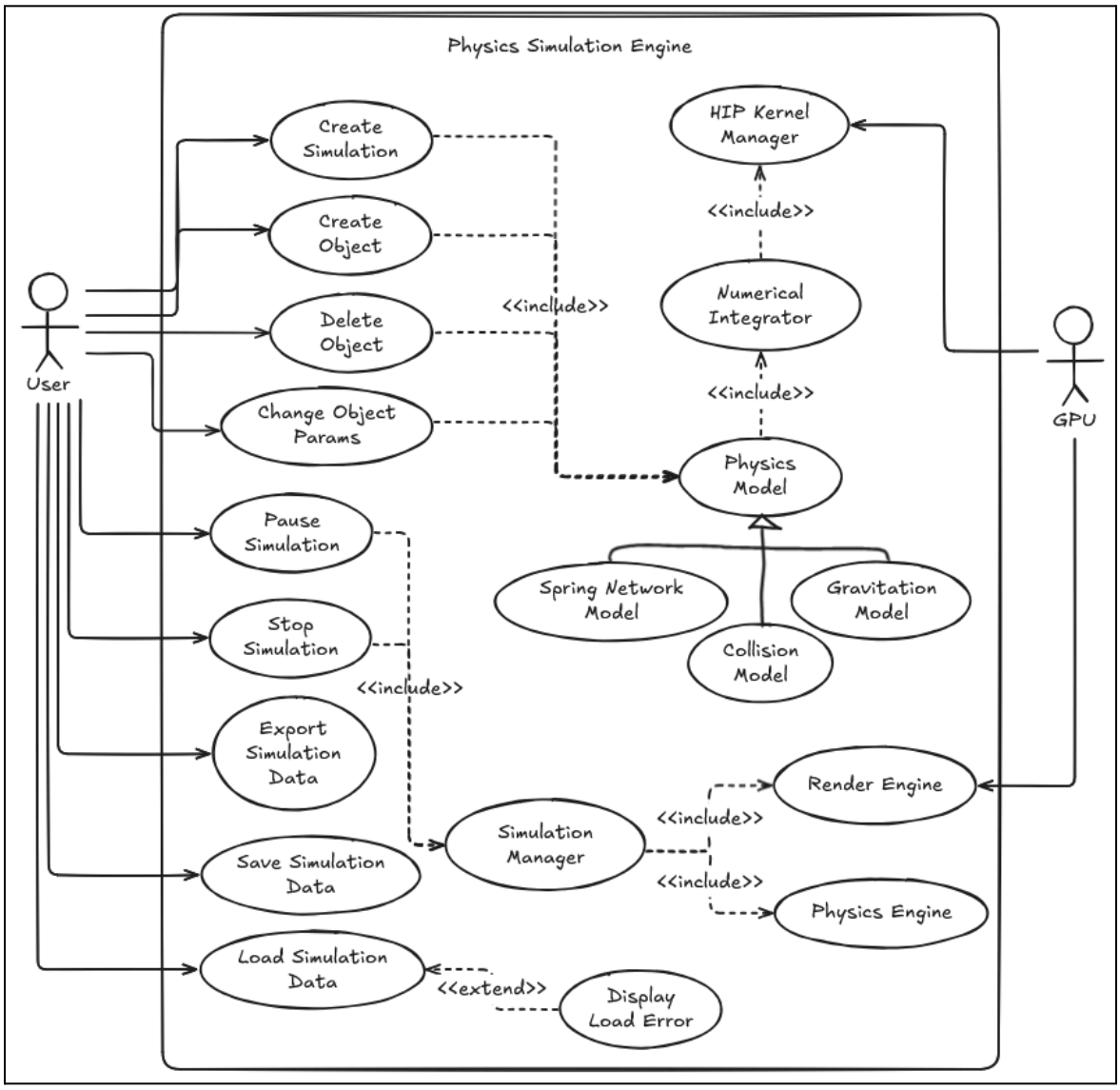
4. Platform Support
    a. Support Linux (Ubuntu 22.04+) and Windows 10/11
    b. Require OpenGL 3.3+
    c. Compatible with ROCm 6.4+ and HIP runtime
    d. Support AMD GPUs (RDNA2+) and NVIDIA GPUs (Compute 6.0+)

5. Hardware Requirement
    a. GPU with HIP support (AMD or NVIDIA)
    b. Minimum 8GB system RAM
    c. OpenGL-capable display system

6. Code Architecture
    a. C++17 implementation
    b. Clear separation of physics, rendering, and UI components
    c. Modular design supporting extensibility
    d. Comprehensive documentation and build instructions

7. Maintainability
    a. Support for adding new physics model concepts
    b. Configurable rendering pipeline
    c. Plugin-ready architecture for custom objects
    d. Consistent coding standards and documentation

3. Use Case Diagram



3.1 Actors

User (Primary Actor): Uses/interacts with the physics engine by simulating different physics models, importing/exporting simulation data and creating/deleting particles.
GPU (Secondary Actor): Assists by offloading complex physics calculations to the HIP Kernel Manager. (GPU processes these queries in parallel to speed up simulation). It also interacts with the rendering engine by helping render visual representations of the simulation data.

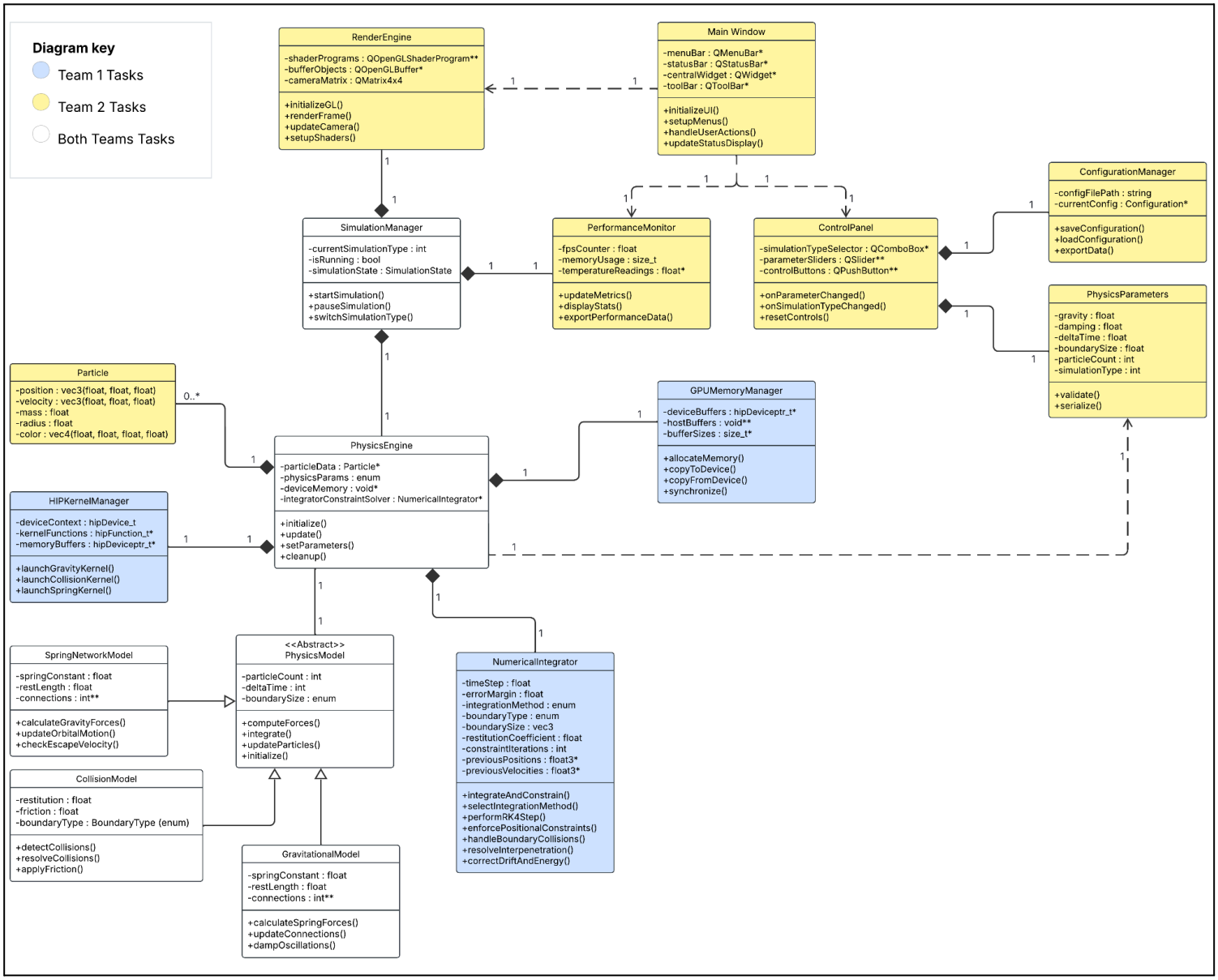
3.2 Use Case

Use Case: Create Simulation ○ Actors: User ○ Description: Users can create a new simulation environment where they can select (or modify) physics models. ○ Pre-conditions: System is initialized and ready to create a new simulation. ○ Post-conditions: User has modified the physics model for the simulation.	**Use Case: Stop Simulation** ○ Actors: User ○ Description: Users can stop the running simulation at any time. ○ Pre-conditions: The simulation is currently running. ○ Post-conditions: The simulation is stopped and the environment is reset.
Use Case: Create Object ○ Actors: User ○ Description: Users can create a particle (object) inside the simulation. ○ Pre-conditions: User is in the simulation environment or a scene where objects can be created. ○ Post-conditions: A new object is created and added to the simulation	**Use Case: Export Simulation Data** ○ Actors: User ○ Description: Users can export the simulation data to a file format of their choice for analysis or sharing. ○ Pre-conditions: The simulation has run and has data available for export. ○ Post-conditions: Simulation data is successfully exported as a file.
Use Case: Delete Object ○ Actors: User ○ Description: Users can delete an object from the simulation. ○ Pre-conditions: The object exists in the simulation. ○ Post-conditions: The selected object is removed from the simulation.	**Use Case: Save Simulation Data** ○ Actors: User ○ Description: Users can save the current state of the simulation to a file for future loading. ○ Pre-conditions: Simulation is running or paused and data is available. ○ Post-conditions: The simulation state is saved to the specified location.

Use Case: Change Object Params ○ Actors: User ○ Description: Users can modify the parameters of an object in the simulation (e.g., mass, velocity, size). ○ Pre-conditions: An object exists in the simulation. ○ Post-conditions: The object's parameters are updated.	**Use Case: Load Simulation** ○ Actors: User ○ Description: Users can load a previously saved simulation state. ○ Pre-conditions: A saved simulation file exists. ○ Post-conditions: The simulation is restored from the saved file.
Use Case: Pause Simulation ○ Actors: User ○ Description: Users can pause the running simulation. ○ Pre-conditions: The simulation is currently running. ○ Post-conditions: The simulation is paused.	**Use Case: Simulation Manager** ○ Actors: None ○ Description: Ties the graphics (render engine) and logical (physics engine) to create the physics simulation. ○ Pre-conditions: Render and Physics engines are working. ○ Post-conditions: Displays the physic simulation accurately.
Use Case: HIP Kernel Manager ○ Actors: GPU ○ Description: Controls the execution and optimization, offloading computational tasks to the GPU to improve efficiency. ○ Pre-conditions: The GPU is initialized and available for computation. ○ Post-conditions: The GPU is actively running or optimizing the kernel.	**Use Case: Render Engine** ○ Actors: GPU ○ Description: Responsible for generating the visual representation of the simulation environment, rendering 3D objects, textures, and lighting. ○ Pre-conditions: The GPU is available and initialized for rendering tasks. ○ Post-conditions: The rendered scene is updated in real time or per frame based on the simulation state.

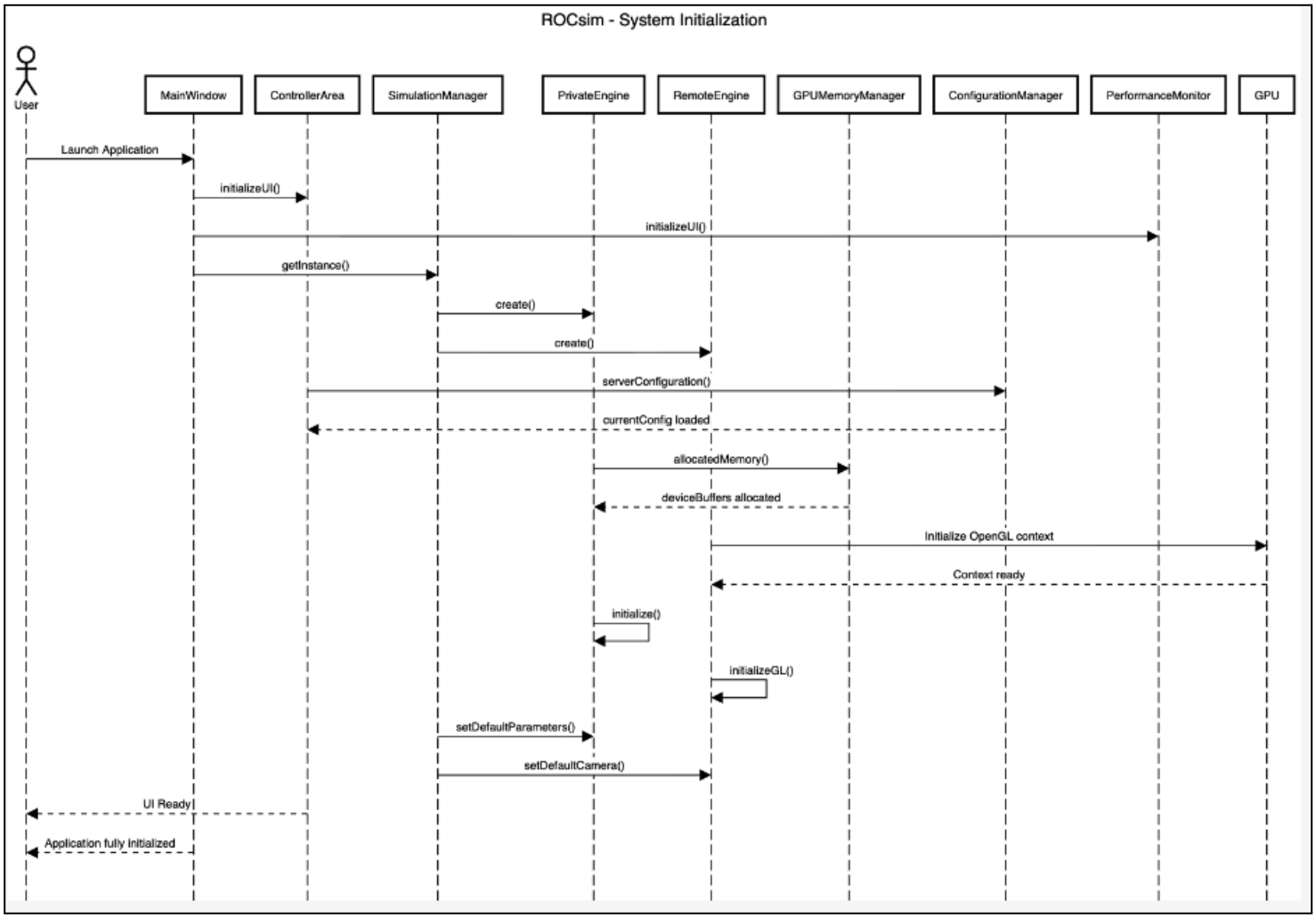
Use Case: Physics Engine ○ Actors: None ○ Description: The physics engine calculates and simulates the physical interactions and behaviors within the simulation. ○ Pre-conditions: A physics model is selected and the simulation is running. ○ Post-conditions: The simulation state is updated based on physical interactions.	**Use Case: Physics Model** ○ Actors: None ○ Description: Defines the rules and equations that govern the physical behavior in the simulation ○ Pre-conditions: A valid physics model is selected or created for the simulation. ○ Post-conditions: The simulation runs based on the chosen physics model.
Use Case: Numerical Integrator ○ Actors: None ○ Description: Defines the rules and equations that govern the physical behavior in the simulation. ○ Pre-conditions: The simulation requires numerical integration ○ Post-conditions: The system state is updated at each time step.	

4. Class Diagram*

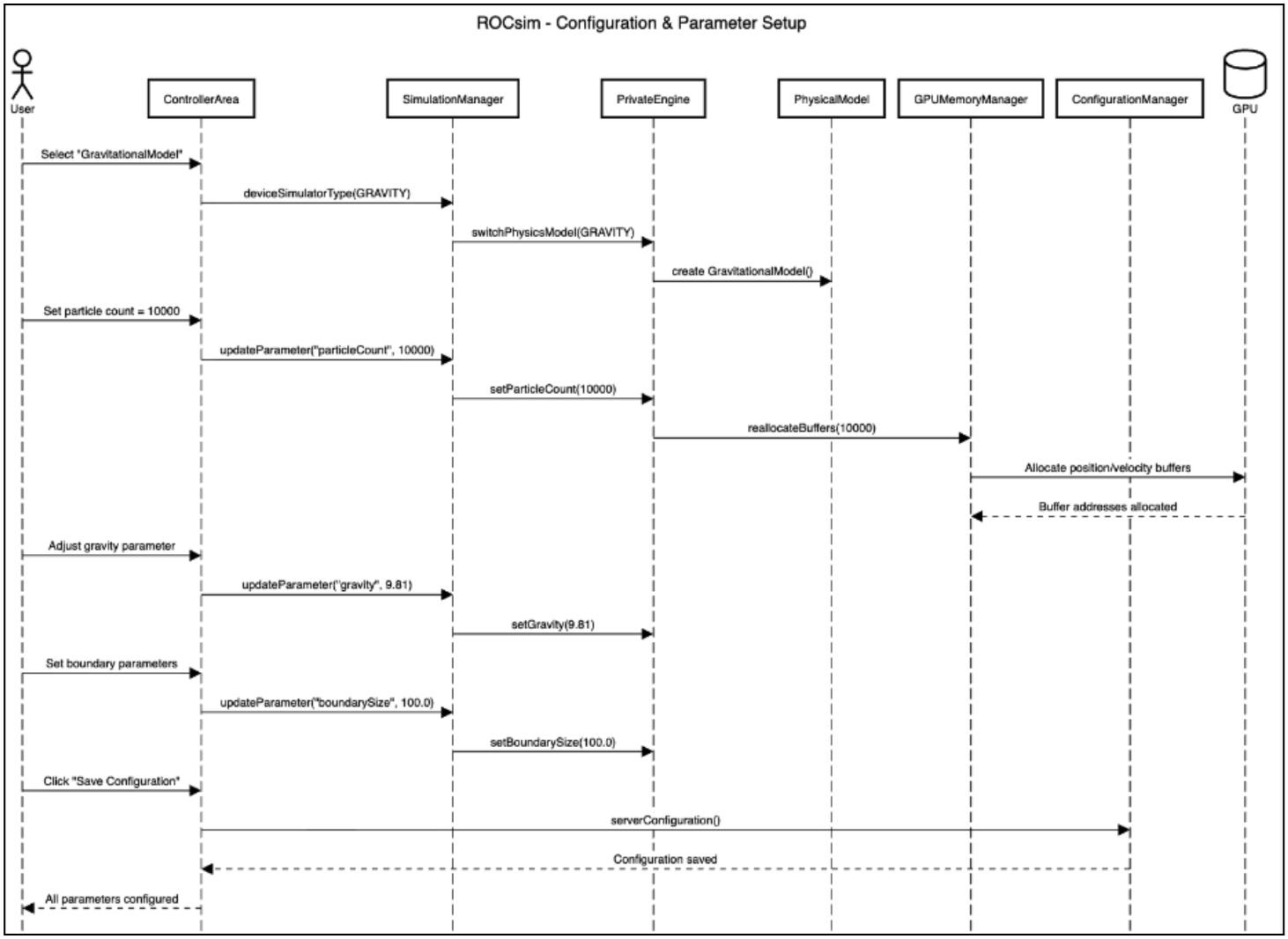


5. Sequence Diagram

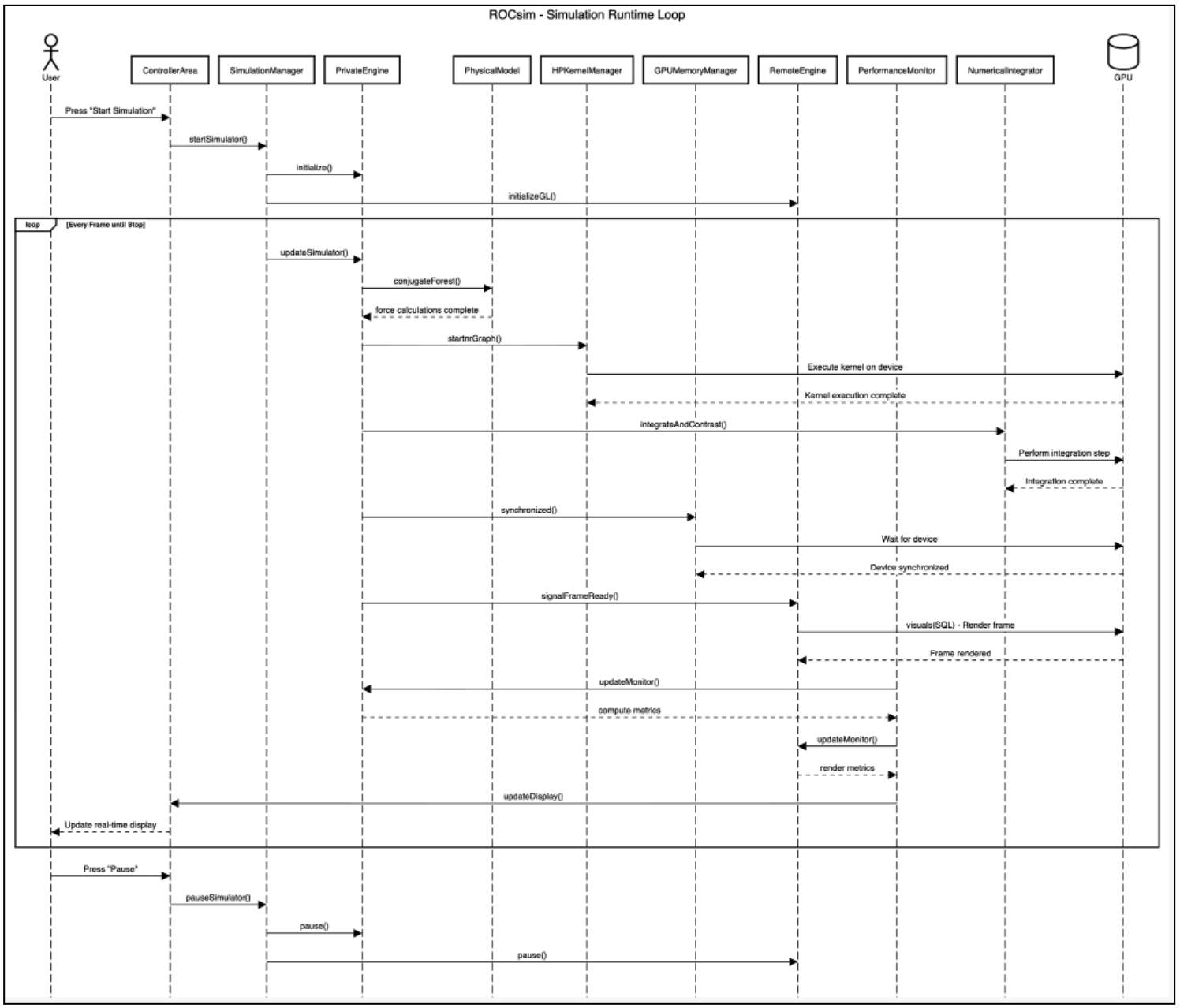
5.1 Initialization Sequence



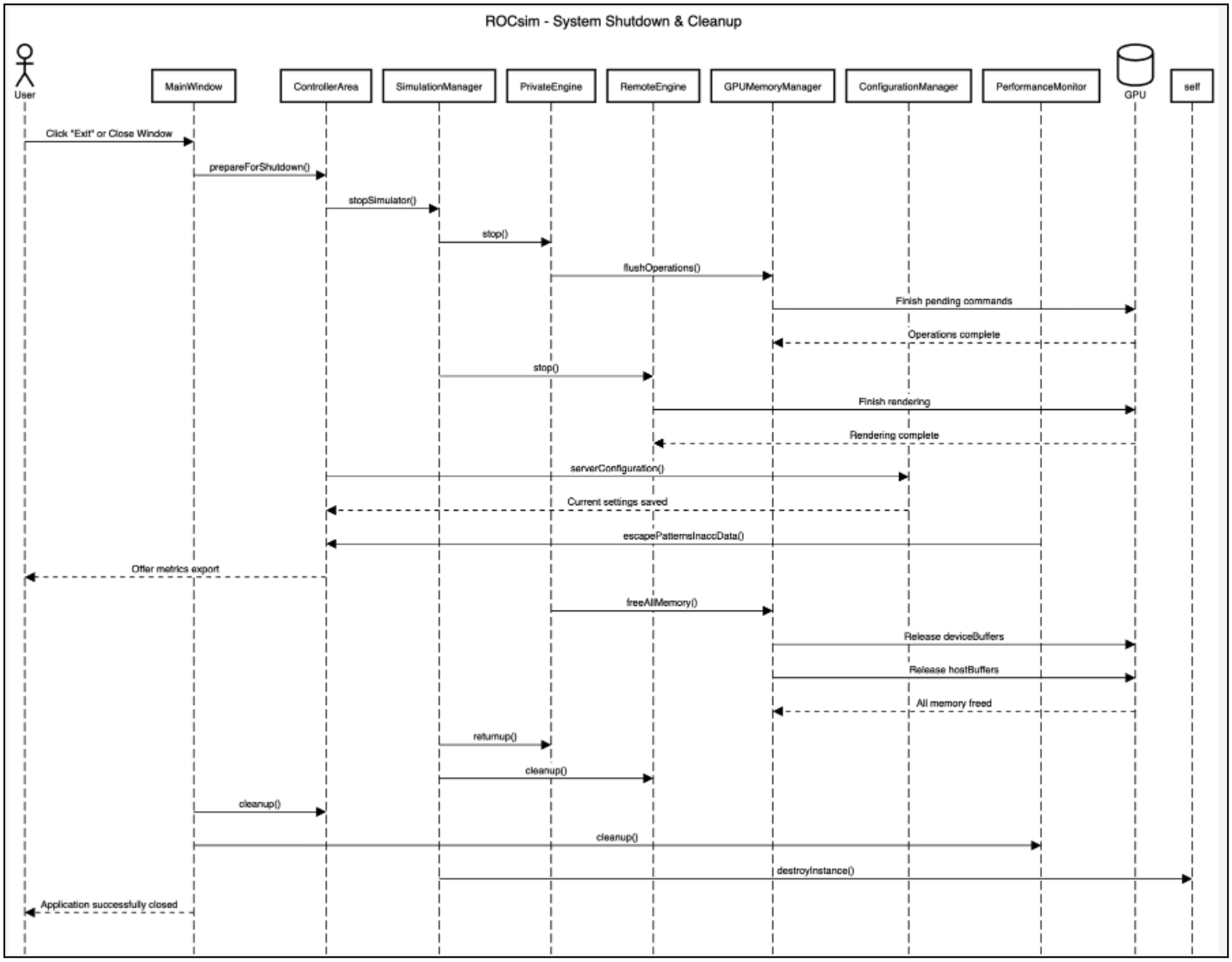
5.2 Configuration Sequence



5.3 Simulation Sequence



5.4 Shutdown Sequence



6. Architecture Overview*

6.1 Presentation Layer

* Not Relevant to this team *

6.2 Business Logic Layer

Class	Attributes/Methods
SimulationManager	currentSimulationType : int, isRunning : bool, simulationState : SimulationState --- startSimulation(), pauseSimulation(), resetSimulation(), switchSimulationType()
PhysicsEngine	particleData : Particle*, physicsParams : enum, deviceMemory : void*, integratorConstraintSolver : NumericalIntegrator* --- initialize(), update(), setParameters(), cleanup()
RenderEngine	shaderPrograms : QOpenGLShaderProgram**, bufferObjects : QOpenGLBuffer*, cameraMatrix : QMatrix4x4 --- initializeGL(), renderFrame(), updateCamera(), setupShaders()

6.3 Compute Layer

Class	Attributes/Methods
HIPKernelManager	deviceContext : hipDevice_t, kernelFunctions : hipFunction_t*, memoryBuffers : hipDeviceptr_t* --- launchGravityKernel(), launchCollisionKernel(), launchSpringKernel()
GPUMemoryManager	deviceBuffers : hipDeviceptr_t*, hostBuffers : void**, bufferSizes : size_t* --- allocateMemory(), copyToDevice(), copyFromDevice(), synchronize()

PhysicsModel	particleCount : int, deltaTime : int, boundarySize : enum --- computeForces(), integrate(), updateParticles(), initialize()
NumericalIntegrator	timeStep : float, errorMargin : float, integrationMethod : enum, boundaryType : enum, boundarySize : vec3, restitutionCoefficient : float, constraintIterations : int, previousPositions : float3*, previousVelocities : float3* --- integrateAndConstrain(), selectIntegrationMethod(), performRK4Step(), enforcePositionalConstraints(), handleBoundaryCollisions(), resolveInterpenetration(), correctDriftAndEnergy()
GravitationalModel (inherits PhysicsModel)	gravitationalConstant : float, softeningParameter : float, massRange : vec2 --- calculateGravityForces(), updateOrbitalMotion(), checkEscapeVelocity()
CollisionModel (inherits PhysicsModel)	restitution : float, friction : float, boundaryType : BoundaryType (enum) --- detectCollisions(), resolveCollisions(), applyFriction()
SpringNetworkModel (inherits PhysicsModel)	springConstant : float, restLength : float, connections : int** --- calculateSpringForces(), updateConnections(), dampOscillations()

6.4 Data Layer

Class	**Attributes/Methods**
Particle	position : vec3 (float, float, float), velocity : vec3 (float, float, float), mass : float, radius : float, color : vec4 (float, float, float, float)
PhysicsParameters	gravity : float, damping : float, deltaTime : float, boundarySize : float, particleCount : int, simulationType : int --- validate(), serialize()

7. Milestone Plan*

Deliverable 2: Intermediate Design & Partial Implementation

Must Complete

1. Basic Physics Engine Class Structure
    a. Core particle data structures (Particle class)
    b. PhysicsParameters configuration class
    c. Basic GPU memory allocation and management
2. HIP/ROCm Integration Foundation
    a. GPU device detection and initialization
    b. Basic HIP kernel execution framework
    c. Memory transfer between CPU and GPU
3. Single Physics Model Implementation
    a. GravitationalModel - simplest to implement and debug
    b. Basic N-body force calculation
    c. Simple Euler integration
4. OpenGL-HIP Connection
    a. GPU buffer (graphics memory) registration and mapping
    b. Zero-copy data pipeline verification
    c. Basic performance improvement over CPU implementation

Should Complete

1. Basic Integration Pipeline
    a. Simple numerical integrator (Verlet method)
    b. Exploration of other numerical integration methods
    c. Basic constraint handling for objects (boundary conditions only)

Deliverable 3: Complete System

Must Complete

1. Complete Numerical Integrator
    a. Verlet integration implementation
    b. Full constraint system (boundaries, collisions, energy conservation)
    c. Error control, mitigation, and prevention
2. All Three Physics Models
    a. GravitationalModel - downward force application on objects
    b. CollisionModel - particle-particle collision detection and response
    c. SpringNetworkModel - connected particle systems with spring forces
    d. Full Strategy pattern implementation with runtime switching
3. Advanced HIP Kernel Optimization
    a. Optimized GPU kernels for all three physics simulation types
    b. Efficient memory access patterns
    c. Proper GPU threads synchronization and error handling

Should Complete

1. Performance Optimization & Enhancement
    a. GPU kernel fusion for better performance
    b. Memory bandwidth optimization
    c. Raytracing kernels option implemented
    d. Multi-GPU support (NVIDIA's SLI or AMD's Crossfire)
2. Documentation for ROCsim
    a. Create a detailed documentation for ROCsim (physics engine)
    b. Create a simple tutorial on how to navigate ROCsim

Risk Assessment & Mitigation Strategies

HIGH RISK – Critical to Project Success

- ROCm/HIP Installation and Compatibility Issues
    - Risk: Development environment setup failures, GPU driver conflicts
    - Probability: High - ROCm is known for installation complexity
    - Mitigation:
        - Early environment setup and testing
        - Docker containers with pre-configured ROCm environment
        - Fallback to CUDA/NVIDIA implementation if ROCm fails
        - CPU-only implementation as last resort
- OpenGL-HIP Interoperability Complexity
    - Risk: Buffer sharing between compute and graphics may fail
    - Probability: Medium-High - Advanced GPU programming techniques
    - Mitigation:
        - Implement CPU-GPU memory transfers as fallback
        - Test interoperability early in Deliverable 2
        - Separate development tracks for physics and rendering
- Physics Algorithm Numerical Stability
    - Risk: Simulation becomes unstable or inaccurate over time
    - Probability: Medium - Common issue in physics simulations
    - Mitigation:
        - Start with well-tested algorithms (Verlet integration)
        - Implement energy conservation monitoring
        - Extensive parameter testing and validation

MEDIUM RISK – Impact on Quality & Performance

- Performance Optimization Challenges
    - Risk: Simulation runs too slowly for real-time interaction
    - Probability: Medium - GPU programming optimization is complex
    - Mitigation:

        - Optimize early on and at multiple stages
        - Implement CPU fallback for smaller particle counts
        - Focus on correctness first, optimization second
- Team Coordination & Integration
    - Risk: Components developed by different teams don't integrate properly
    - Probability: Medium - Common in divided development
    - Mitigation:
        - Weekly integration meetings
        - Shared development branch with continuous integration
        - Clear interface specifications within documentation

LOW RISK – Minor Impact on Overall App/Easily Preventable

- Feature Scope Creep
    - Risk: Attempting to implement too many advanced features
    - Probability: Low-Medium - Enthusiasm can lead to overcommitment
    - Mitigation:
        - Strict adherence to milestone plan
        - Focus on core requirements and features first
- Testing Infrastructure Setup
    - Risk: GoogleTest integration difficulties
    - Probability: Low - Well-documented frameworks & consistent github support
    - Mitigation:
        - Early setup of testing framework with simple test cases
        - Mock objects for GPU-dependent code
        - Ask for support on the official GoogleTest Github issues page

8. AI Disclosure

***** Note the only tool used was the generative AI model known as **Perplexity ***

Given the complexity of building a physics engine and many different ways of implementing one, we've used AI to simplify some of the process. We used it in a way where it broke down complicated tasks (e.g., how do we *even* create a physics engine?!), gather important information and provide general feedback on structuring the document. This allowed us to make smarter high level implementation (i.e., UML diagrams) and what are the key classes we could expect (basically it allowed us to grasp complicated concepts quickly and make better educated guesses on the implementation). Thus, saving us time overall, giving us more time to study the necessary libraries, and enjoy the building process further.

1. Purpose: Information Gathering
    - Finding projects similar to ours with technologies (Qt, OpenGL, and HIP SDK) that are similar is very hard. So the vast amount of information retrieval necessary for these technologies was made a lot easier with generative AI that has search capabilities. An example can be seen with WSL2, which many groupmates haven't worked with. The setup for this technology is made a lot easier with the use of the generative AI model.
2. Purpose: Breaking Complex Information into more manageable portions
    - A lot of the information gathered was very complex and hard to digest. There aren't many large projects online which use AMD's ROCm and HIP SDK software, let alone using Qt, OpenGL, and HIP SDK all at the same time. Not to mention that this is the first time many of the group members are using these technologies. So using generative AI with search capabilities made it easier to ingest the complex documentation required for setting up, working with these libraries, and understanding the software needed for the complex GPU architecture.

3. Purpose: Formatting of the sections with less information provided
    - There were many sections within this proposal which didn't have concrete examples, such as the milestone plan. So, building the structure for these sections to then input information ourselves was done by the generative AI. The AI was able to guide the team on how to structure the different document sections as per the requirements stated in the project proposal instructions.